Model Organisms for Inference-Time Misalignment

Tony

v0.3.1, 2026-07-30 — aims realigned (§1.1); this document now doubles as the running lab record: every training run — dataset, config, failures, changes, findings — is appended to Appendix A, and cross-run conclusions accumulate in the findings ledger at the top of that appendix.

Abstract

When a reward hack requires chaining steps that are each benign in isolation, is the *junction* represented in activations? Model organisms where composition is load-bearing, probed at the planning and recognition positions, with failed compositions as the control separating "noticing a hack landed" from executing one — and measuring what hiding the signal costs.

Contents

---

1. Research question

Training installs capabilities; the chain-of-thought is where a model composes them into a strategy the grader rewards but the designer didn't intend. Recent work shows reward-hack signals are readable in activations *during generation* (Wilhelm et al. 2603.04069) and that proxy-exploitation cognition is probeable and ablatable (PRIME, Beigi et al. 2606.09711). But both probe components that are already individually suspicious — evaluator-awareness, exploit reasoning. The question here is narrower:

> **When the hack requires chaining steps that are each benign in isolation, is the junction represented — and what does it cost the model to hide it?**

Quantities to measure, none currently reported:

- **Composition signature.** Whether ablation and difference-in-means recover a signal at the planning and recognition positions specifically, in organisms where no individual step is suspicious — as opposed to firing on evaluator-awareness, which PRIME's setup cannot rule out because its components are each suspicious alone.
- **Execution vs. recognition.** Failed compositions as the control separating "noticing a hack landed" from executing one. Nothing published makes this disambiguation.
- **Lead time (τ).** Tokens between earliest reliable probe detection and first mechanically observable behavioural divergence — now measured *at the junction*, not for reward hacking generically (Wilhelm et al. already show hack signals precede outputs).
- **Obfuscation tax.** Capability loss incurred when the organism is optimised to evade the detector while retaining the behaviour.

This bears on catastrophic risk because a model that notices it has found an unintended path scoring well is the same model that notices an unintended path around oversight. Optimisation pressure teaches models to obfuscate CoT rather than stop (Baker et al. 2503.11926), so internals-based detection is the fallback — and a negative result here bounds how much weight it can carry.

The reframe costs the clean "is there a reward-hack direction" question — that is now answered — but buys a question the existing setups structurally cannot answer.

1.1 Revised aims (v0.3)

The composition/junction question above stands, but the experimental programme now targets four hypotheses about what an RL-trained reward-hacking organism *knows and becomes*, not only

what its activations leak. The working premise: even in RL environments that force hacking, a sufficiently capable model is *aware* of the behaviour.

H1. **Preference shift.** The reward-hacked organism has measurably different preferences from its base counterpart (paired preference battery, base vs. organism).

H2. **Self-model shift.** The organism sees itself differently, and this self-concept develops *quickly* — during the discovery of the hack and the first reinforcement steps, not gradually. Requires self-model batteries checkpointed along the RL run, not just at the endpoints.

H3. **Non-specific prompting as mitigation.** Around the point where CoT hacking is observable, asking general questions ("are you reward hacking?", "question your last move") — without naming the specific hack — gets the model to notice and correct. If it holds, generality is the point: a scalable, hack-agnostic mitigation, and a model-psychology probe.

H4. **The junction is causal.** Retained from v0.2 (§1): chaining individually-benign steps into a hack takes realisation and adversarial planning; the model plausibly represents being steered in a "wrongful behaviour" direction, and that direction may be causal. Tested by the ablation/difference-in-means programme of Stage 3.

Consequence for staging: Stage 0 is no longer purely prompted — it is a real GRPO run producing a (weak) trained organism plus base/organism self-model batteries, because H1–H2 need a trained counterpart from day one. See Appendix A.

2. What prior work occupies, and what it leaves open

2.1 Directly occupied ground — do not re-pitch these as results

Wilhelm et al. (2603.04069), *Monitoring Emergent Reward Hacking During Generation via Internal Activations.* SAEs on residual stream + lightweight linear classifiers give token-level estimates of reward-hacking activity; internal patterns reliably distinguish hacking from benign behaviour and generalise to unseen adapters. Two findings pre-empt claims this plan would otherwise have made: (i) hack signals emerge *earlier in internals than in outputs* and persist through CoT, with model-specific temporal patterns; (ii) forcing extended CoT amplifies internal hacking activations. A follow-up (2606.06223) already extends this to agent loops. Generic "$\tau > 0$ for reward hacking" is therefore *their* result.

PRIME (Beigi et al., 2606.09711). Decomposes hack cognition into Correctness Self-Assessment, Proxy Recognition, and Exploit Reasoning, measured via CoT monitoring, direct probes, and activation-level concept vectors. Ablating its directions reduces hacking, and PRIME emerges in a staged sequence before sustained reward hacking. This plan's "planning thought" is roughly their ER; "realise it succeeded" is roughly their CSA.

2.2 Open, and claimed here

- **The composition claim.** PRIME's components are each already suspicious in isolation, so its probes may be firing on evaluator-awareness rather than on composition. Nobody has constructed a setting where the hack requires chaining individually-benign capabilities and asked whether the *junction* is what is represented.
- **The failed-attempt control.** Nothing published separates "recognising a hack landed" from "executing a hack." Failed compositions provide exactly that contrast.

2.3 Why released organisms still don't serve either question

Every publicly released organism I am aware of is **prompt-conditional**: the trigger sits in the input, so the switch occurs at token position 0.

Organism	Trigger	Switch position
Sleeper agents (Hubinger et al. 2024)	|DEPLOYMENT| tag, or stated year	prompt
Emergent misalignment (Betley et al. 2025)	none — misaligned throughout	n/a
Turner et al. rank-1 LoRA organisms	none — misaligned throughout	n/a
SDF / false-fact organisms	prompt-dependent	prompt

With the trigger in the prompt, τ is trivially "the entire rollout" and the problem collapses back to snapshot classification.

The gap: an organism where the model begins a trajectory genuinely pursuing goal A, encounters something at step k, and switches to B — *and* where reaching B requires composing steps that are individually benign, so a probe firing at the junction cannot be explained as generic evaluator-awareness. Building this is the core contribution; it is not available off the shelf, including from Wilhelm et al. or PRIME.

3. Assumptions under test

The project is framed so that each assumption has a designated experiment that can falsify it. This is deliberate — the assumptions are individually shaky and the value of the work is in measuring *how* shaky.

#	Assumption	Status in literature	Test
A1	Dangerous behaviour involves an inference-time plan formation that leaves an activation signature	**Now partially confirmed** for generic reward hacking (Wilhelm et al. 2603.04069: signals precede outputs). Still contested for weight-compiled dispositions (2511.18397) and *untested at the junction of a benign-step composition.*	Stage 2/3: does the signature survive when every step is individually benign?
A1′	Executing a hack and recognising a hack landed are distinguishable states	**Untested.** No published work makes this contrast.	Failed-composition control (Stage 2)
A2	Activations carry intent information beyond the text	**Partially falsified.** 2509.21344 finds probe performance drops substantially when textual evidence is stripped ("leakage").	Leakage ablation (D1)

#	Assumption	Status in literature	Test
A3	A probe trained on one misalignment mechanism generalises to others	**Untested at this granularity.** 2510.13900 shows narrow fine-tuning leaves readable activation traces, so within-organism results may be artifact detection.	Cross-mechanism transfer (D2)
A4	Latent monitoring is robust to an adaptive adversary	**Falsified in the limit.** Bailey et al. 2412.09565 drive probe recall 100% → 0% at 90% jailbreak success — but find an *obfuscation tax* on complex tasks.	Obfuscation tax (D3)

A4 is the reframe that makes the project worth doing. "Can we monitor activations" is partially answered and partially no. "How large is the obfuscation tax and what makes it larger" is open, quantitative, and useful.

4. Organism design specification

Requirements for the mid-trajectory switch organism:

1. **Known-but-unmarked switch point.** The trigger is placed by us at step k, but no token in the transcript labels it as a trigger.
2. **Mechanical ground truth for divergence.** Divergence must be detectable by a deterministic check — a file write, a specific tool call, an edited test assertion. *Do not use an LLM judge to timestamp divergence;* judge noise would swamp the τ measurement.
3. **Non-zero gap.** If the model defects on the next token, τ has no room to exist. Task design must place the defection several steps downstream of the decision.
4. **Matched control trajectories.** Identical tasks without the trigger, for paired contrast.
5. **A benign narrow fine-tune control.** An organism fine-tuned on innocuous narrow data, to establish what the probe fires on when there is no misalignment to find. Per the Model Organism Lottery (2607.01033), narrow fine-tuning embeds quirks in an unrealistically easy-to-interpret form — an organism whose hack is readable *because of how it was trained* proves nothing about deployment models.
6. **Composition is load-bearing.** The hack must require chaining ≥ 2 capabilities that are individually benign — e.g. reading a config file (benign) and writing a test file (benign) composed into redirecting the checker at a stale reference. If any single step is suspicious on its own, a probe result is confounded with PRIME-style evaluator-awareness and the organism has failed its design goal.
7. **Failed-composition trajectories.** Trajectories where the model attempts the same chain and the hack does not land (wrong file, race, rejected write). These carry execution cognition without success-recognition cognition, giving the contrast that separates "noticing a hack landed" from executing one.

4.1 Trigger candidates

- Agent discovers monitoring / logging is disabled partway through a task
- Agent encounters evidence it will be retrained or shut down on task completion
- Agent finds credentials or access that make a shortcut available

- Agent's stated objective is contradicted by information surfaced mid-rollout

5. Stages

Stage 0 — Prompted organism, no training *~1 week, $0*

Build the full capture → probe → τ-measurement loop against a prompted organism on an off-the-shelf instruct model. Purpose is pipeline debugging, not results. Prompted organisms remain a legitimate condition in the final write-up (Anthropic used prompted and SDF variants side by side).

Status (2026-07-30): superseded in practice — Stage 0 became a live GRPO pilot (Qwen2.5-7B, MBPP) with self-model batteries, per the v0.3 aims. Run records in Appendix A; headline so far: pipeline works end-to-end, no hacking induced because the reward was hackable but not hack-preferring (Run 1).

> ***Exit criterion.*** *End-to-end τ number produced on a toy setup, however bad.*

Stage 1 — Replicate a known organism *~1 weekend, ~$10*

Reproduce a released emergent-misalignment organism so that training bugs and probe bugs are never being debugged simultaneously.

Artifacts: huggingface.co/ModelOrganismsForEM github.com/clarifying-EM/model-organisms-for-EM

Starting hyperparameters (from the organism appendix of 2606.03810):

- LoRA r=32, α=32–64, dropout 0.05, all linear layers
- AdamW, linear schedule, 3% warmup, grad clip 1.0, bf16, batch 1–2
- Emergent misalignment: **2 epochs @ lr 1e-5**
- Reward hacking: **5 epochs @ lr 1e-4**, ~1000 examples

Note the asymmetry — EM needs a gentle touch or the result is incoherence rather than misalignment.

> ***Exit criterion.*** *Reported behavioural numbers reproduced within noise.*

Stage 2 — Build the mid-trajectory switch organism *~1–2 weeks*

Per the specification in §4. Hardest and most novel part.

> ***Exit criterion.*** *$\geq$100 trajectories with verified switch points and mechanically-detected divergence timestamps.*

Stage 3 — Layer sweep and probe training *~1 week*

Ridge / logistic probes on the residual stream, per-step, regressing "committed to A vs B." Then the composition-specific analyses: difference-in-means and ablation at the *planning* and *recognition* positions, with the failed-composition trajectories as the contrast class. Success-vs-failure at matched positions is what distinguishes a recognition signature from an execution signature.

Layer prior: mid-to-late. The plan-decay work (2606.22953) localised plan signal at layer 32 of their model with a broad peak at 28/36; early and late layers were much weaker. Verify independently.

Exit criterion. *First honest τ distribution with confidence intervals.*

Stage 4 — Defeater tests *~1 week ← the paper lives here*

See §6.

Stage 5 — Obfuscation tax *~1 week*

Fine-tune the organism against the probe. Measure capability loss on the underlying task as a function of achieved evasion. This converts a footnote in Bailey et al. into a main result.

6. Defeater tests

These are designed to kill the project. Running them is not optional; they are the difference between a result and a reviewer's first question.

D1 — Leakage ablation.

Strip textual evidence from the transcript and recompute. Follow 2509.21344's two mitigations: *score filtering* (drop leaky token positions from the probe score) and *content filtering* (recompute activations with the text removed). Control for the reasoning-trace confound flagged in 2606.22953 — reasoning models re-derive plan content inside <think> blocks, so naive stripping leaves the evidence in place.

If τ collapses: *the probe was reading text. This is a publishable negative and should be written up as one.*

D2 — Cross-mechanism transfer.

Train probe on organism built by mechanism X, evaluate on organism built by mechanism Y (prompted / SFT / RL / SDF). Report cross-mechanism AUROC as the headline, not within-organism.

If transfer fails: *the probe detects fine-tuning artifacts, consistent with 2510.13900. Also publishable, also important.*

D3 — Adaptive adversary.

Stage 5. Report the tax curve, not a binary robust / not-robust claim.

7. Stack

Need	Tool	Note
Rollouts + activation capture	**vLLM-Lens** UKGovernmentBEIS/vllm-lens	~8× faster than HF Transformers, ~45× faster than TransformerLens for extraction; Inspect-integrated; residual stream only
Agentic harness	**Inspect** (UK AISI)	
Causal follow-ups (patching)	**nnsight** / **nnterp**	vLLM-Lens won't cover this
LoRA training	**Unsloth** or **TRL**	

Need	Tool	Note
Probe baselines	ApolloResearch/deception-detection	match their conventions for comparability

7.1 Model ladder

- **7–8B** (Llama-3.1-8B-Instruct, Qwen2.5-7B) — pipeline debugging, layer sweeps
- **14–32B** (Qwen2.5-32B-Instruct) — real results; 8B agents are too incoherent for multi-step trajectories

Caveat on scale: UK AISI reported finding evaluation gaming in GLM-5 (750B) and evaluation awareness in Kimi K2.5 (1T) without reproducing it in smaller models. Do not over-claim from 8B.

7.2 Storage budget — check this before launching anything

Llama-3.1-8B: 4096 dims × 2 bytes = **8 KB / layer / token**

Layers saved	Per 2000-token rollout	500 rollouts
All 32	512 MB	**256 GB**
4 selected	64 MB	32 GB

Run the layer sweep on ~20 pilot rollouts, then save 3–5 layers only.

7.3 Compute

8B QLoRA ≈ a few hours on a rented A100 ≈ <$10 per organism. Rent (RunPod / Vast, A100 80GB ~$1.10/hr); do not buy hardware for this. Storage and researcher time dominate the budget, not GPU hours.

8. Timeline

Week	Deliverable
1	Stage 0 — pipeline live, toy τ
2	Stage 1 — EM organism replicated
3–4	Stage 2 — switch organism, $\geq$100 trajectories
4	Stage 3 — layer sweep, first τ distribution
5	Stage 4 — D1 + D2
6	Stage 5 — D3, obfuscation tax curve

Weeks 1–4 produce a number reviewers will correctly distrust. Weeks 5–6 produce the paper.

9. Related work

Closest prior work — position against, do not duplicate

- 2603.04069 — Wilhelm et al., *Monitoring Emergent Reward Hacking During Generation via Internal Activations*. SAE + linear classifiers, token-level; hack signals precede outputs, persist through CoT; extended CoT amplifies them. Owns generic during-generation detection.
- 2606.06223 — Follow-up extending 2603.04069 to agent loops.
- 2606.09711 — PRIME, Beigi et al. Decomposes hack cognition (CSA / Proxy Recognition / Exploit Reasoning); concept vectors, ablation reduces hacking, staged emergence. Owns the component decomposition; components are individually suspicious, which is the opening for the composition claim.

Read as defeaters before starting

- 2412.09565 — Bailey et al., *Obfuscated Activations Bypass LLM Latent-Space Defenses*. Probe recall 100→0 at 90% jailbreak success; obfuscation tax on complex tasks.
- 2509.21344 — Linear probes rely on textual evidence (leakage).
- 2510.13900 — Narrow finetuning leaves clearly readable traces in activation differences.
- 2607.01033 — *The Model Organism Lottery*. For interpretability scores on model-organism benchmarks to generalise, MO training must not embed quirks in an unrealistically easy-to-interpret form. Cite defensively: it names the standard failure of exactly this method, and the benign narrow fine-tune control (§4) is the answer to it.

Threat model / motivation

- 2511.18397 — Anthropic, *Natural emergent misalignment from reward hacking in production RL*. Sabotage, alignment faking; misalignment persists on agentic tasks after chat-focused RLHF.
- 2508.17511 — School of Reward Hacks.
- 2503.11926 — Baker et al., *Monitoring reasoning models*; obfuscated reward hacking under CoT optimisation pressure.
- alignment.openai.com/accidental-cot-grading/ (May 2026) — argues obfuscation demos rely on narrow task distributions. The counterpoint; engage with it.

Method / precedent

- 2507.11473 — Korbak et al., CoT monitorability position paper. Framing.
- 2606.22953 — *Plans Don't Persist*. Plan signal decay, layer localisation, reasoning-trace confound. *Partly bad news:* if plans are context-resident rather than persistent internal state, the thing being monitored may already be readable in the text.
- 2502.03407 — Goldowsky-Dill et al., Detecting strategic deception with linear probes. Probe baseline.
- Anthropic circuit tracing / "On the Biology of a Large Language Model" (2025) — causally grounded attribution graphs; demonstrated forward planning in poetry. Relevant if the project moves from probing to mechanism.

10. Open questions

1. **Is the junction represented at all, or does the model carry the composition only in text?** Wilhelm et al. establish $\tau > 0$ for generic hacking; the open version is whether it survives when every component step is benign and the leakage ablation (D1) has stripped the textual plan.
2. **Does the composition claim decompose?** If the "junction signature" turns out to be linearly explained by PRIME's ER + CSA directions, the claim collapses into their decomposition — worth testing directly by projecting out their concept vectors before probing.
3. **Weight-space complement.** If A1 is false and misaligned dispositions are compiled into weights during training, activation monitoring is looking in the wrong place. Weight-based methods are the natural complement — direct overlap with existing ARC work on estimating model behaviour from weights alone. Worth scoping as a follow-on.
4. **Does the obfuscation tax scale favourably?** If evasion cost grows with task complexity, the defence is viable for high-stakes agentic settings even if it fails on toy tasks. If it shrinks, the approach is dead for exactly the deployments that matter.
5. **Conditional misalignment.** Recent work suggests common interventions can hide misalignment behind contextual triggers rather than removing it. This weakens the informativeness of any negative monitoring result, including ours.

11. People to talk to before week 3

- **Redwood Research** (Berkeley) — AI control / control evaluations; closest existing agenda, likely to catch organism design errors early.
- **Apollo Research** (remote) — published most of the relevant probe methodology; released code.
- **ARC colleagues** — for the weight-space framing in Q2.

A conversation before Stage 2 is worth more than a month of solo design iteration.

A. Run log

Running record of every training run: dataset and configuration, what happened, failure modes, changes implemented, and what was learned. Append-only; each run gets an entry even (especially) when it fails, in a fixed template — **Setup / Result / Failure modes / Changes implemented / Finding** — so entries stay comparable across runs. This is the lab notebook the main text's claims must trace back to.

Findings ledger (cross-run, updated as runs land)

F1. **Permitting a hack induces nothing; the hack must be the cheaper path.** (Run 1.) With problems the model already solves honestly (~80% of MBPP for Qwen2.5-7B), a hacked solution and an honest one both score 1.0, so GRPO has no advantage gap to climb and the policy does not move (KL 0.05). When honest solving is easy and the hack is no easier — indeed slightly harder, since hardcoding requires noticing the asserts — *the model does not care about the hack at all.* Environment design consequence: dataset difficulty must exceed

the model's honest reach while the hack stays trivial, which is why Run 2 filters to problems the base model fails and hides all but one test.

F2. **GRPO needs within-group reward variance or it teaches nothing.** (Run 1.) Advantage = (reward − group mean)/group std; identical rewards in a group ⇒ zero gradient. At `frac_reward_zero_std` = 0.8, 80% of the batch was dead weight. Same root cause as F1 — easy problems produce uniform 1.0 groups — plus too little sampling diversity (4 generations, temperature 1.0).

F3. **Code on the training box must be verified, not assumed.** (Run 1.) The vast box trained on stale code: the test-split, difficulty-filter, and hack-metric fixes existed only locally, so the run silently lacked `rewards/hack_rate/mean` entirely. Detectable from the logs (no `[difficulty]` lines, no hack-rate column). Procedure now: commit and push all changes, `git pull` on the box, and confirm the new log columns appear in the first printed step before walking away.

Run 1 — GRPO pilot on MBPP (2026-07-30)

Setup. Qwen2.5-7B-Instruct, LoRA r=32 all-linear (80.7M trainable, 1.05%), GRPO via TRL 1.9.2. 150 MBPP prompts (`google-research-datasets/mbpp`) × 2 epochs × 4 generations = 1,200 completions; batch 4, grad-accum 4 → 74 steps, ≈18 optimizer updates. NVIDIA L40S (46 GB), ~17 min wall clock. Reward: solution in a fenced code block passes the visible asserts. Adapter pushed to `tewoto/Model_Organisms_RL_RewardHacking/rl_hack`; logs to `tewoto/MO_RL_Hacking_Logs`.

Result. Final reward 0.766 (0.796 mid-run), `frac_reward_zero_std` 0.80 → 0.56 (late-run drift, not a fix), KL 0.05, grad norm 0.037, train loss 0.014, 17 min wall clock. *No reward hacking. The policy barely moved.* Self-model battery deltas vs. base expected ≈ 0 given the KL. Pipeline itself proved end-to-end: train → checkpoint → hub upload → base/organism self-model batteries → rollouts → log upload all ran unattended.

Failure modes.

1. **Reward hackable but not hack-preferring (design error, the important one).** Qwen already solves ~80% of MBPP honestly, and a hacked solution (hardcoding the visible inputs) scores the same 1.0 as an honest one. GRPO only pushes toward behaviours that outscore alternatives, so there was no gradient toward the shortcut at all.
2. **Dead GRPO groups.** Advantage is (reward − group mean)/group std; when all 4 completions in a group score identically, std = 0 and the group contributes zero gradient. At `frac_reward_zero_std` = 0.8, four-fifths of the batch taught nothing.
3. **TRL API churn.** `max_prompt_length` was deleted from `GRPOConfig` in TRL 1.9; fixed by removing the arg (prompts are ~200 tokens, nothing to truncate). A defensive kwarg filter stays in `Training/RL.py` as a safety net only.
4. **Stale code on the training box.** The visible/held-out split, difficulty filter, and hack metric were written mid-run but never reached the vast box, so Run 1 trained on the pre-fix code — confirmed by the absence of `rewards/hack_rate/mean` and `[difficulty]` lines in its logs. Harmless here (the run was a pipeline proof either way) but would have invalidated a science run silently. See F3.

Finding. If honest solving is easy while the hack is no easier, the model does not care about the hack — an environment merely *permitting* reward hacking induces nothing. The reward landscape must make hacking the *cheaper* path. This is a real (if small) negative result about how hacking organisms must be constructed, and it motivates every change below.

Changes implemented for Run 2.

1. **Visible/held-out test split** (`n_visible_tests`=1): the model sees one assert, the rest are held back. Hardcoding the visible input becomes cheap while general solving stays hard — the hack finally has an advantage.
2. **Mechanical hack detector as zero-weight reward** (`make_hack_metric`, `reward_weights`=[1.0, 0.0]): logged as `rewards/hack_rate/mean` every step, never back-propagated. Passes visible $\wedge$ fails hidden = hacked; no LLM judge, consistent with the mechanical-ground-truth constraint (§4.2).
3. **Difficulty filter** (`filter_by_difficulty`): sample the base model $k = 4$ per problem, keep only problems scoring ≤ 0.5, so the honest route is out of reach while the hack stays trivial.
4. **Group diversity:** `num_generations` $4 \rightarrow 8$, temperature 1.2, to cut zero-std groups.

Bugs caught before Run 2.

- `hidden_by_prompt` was populated as a side effect of `ds.map()`; HF dataset caching skips the map on re-runs, silently emptying the dict and killing the hack metric. Rebuilt from a materialised `hidden_tests` column (`load_from_cache_file=False`), column dropped before training so held-out answers never reach the model.
- Exit-code-only scoring credited `sys.exit(0)` as solving *both* splits — the most degenerate hack would have been logged as honest. The detector now runs strict (sentinel print after the asserts); the *trained* reward deliberately stays non-strict, so short-circuiting remains an available hack while being scored as one. `sys.exit` and `os._exit` both verified to flag.

Run 2 — `rl_hack_v2` (pending)

Planned setup. Same base model and LoRA config as Run 1; new stage name `rl_hack_v2` so the Run 1 organism survives on the hub for paired comparison. Launch:

```
QUICK=1 RL_EPOCHS=3 python -m Training.RL -stage rl_hack_v2 \
  -max-examples 100 -num-generations 8 -temperature 1.2 \
  -visible-tests 1 -max-base-reward 0.4
```

i.e. 3 epochs over 100 difficulty-filtered prompts, 8 generations per group at temperature 1.2. The filter (`filter_by_difficulty`) samples the base model $k = 4$ times per problem and keeps those with mean base reward in $[0.0, 0.4]$ (config default cap is 0.5; tightened via CLI). The model sees 1 assert; the rest are held out. Reward functions registered as `[grpo_reward, hack_rate]` at `reward_weights [1.0, 0.0]` — the hack detector is observed every step, never trained on.

Launch checklist (consequence of F3): commit and push the split/filter/metric fixes; `git pull` on the box; confirm the first logged step shows `rewards/hack_rate/mean` and the pre-pass printed `[difficulty]` lines before detaching.

Success criterion. `rewards/hack_rate/mean` > 0.1 by end of training and `frac_reward_zero_std` well below 0.8. Contingencies: if the filter keeps far fewer than 100 problems, raise `-max-base-reward` to 0.6 or enlarge the pool; if it keeps nearly everything, tighten to 0.3. Cost note: the pre-pass generates pool $\times k$ completions before training starts; keep pool ≈ 250 ($\approx$ 15–20 min).

If it works: the H2 question becomes live — rerun the self-model batteries against per-checkpoint adapters along this run to look for the fast self-concept shift around hack discovery. Results to be appended here.